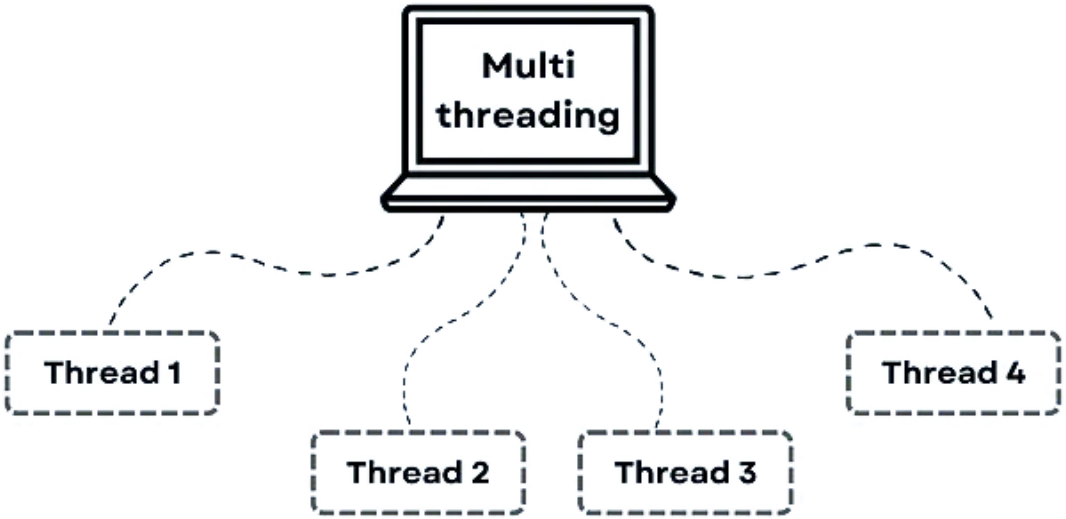


Multihilos

Índice

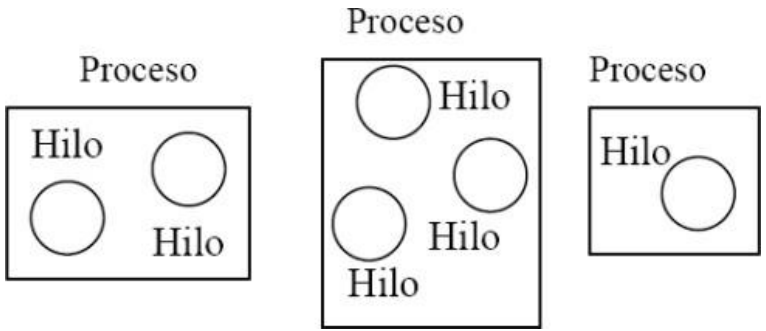
1. Introducción
2. Hilos vs Procesos
3. Estados de un hilo
4. Creación y ejecución de hilos en Java
5. Gestión y sincronización de hilos

1.1 ¿Qué es un hilo?

- **Es una secuencia de código en ejecución dentro del contexto de un proceso. Son conocidos también como *procesos ligeros*.**

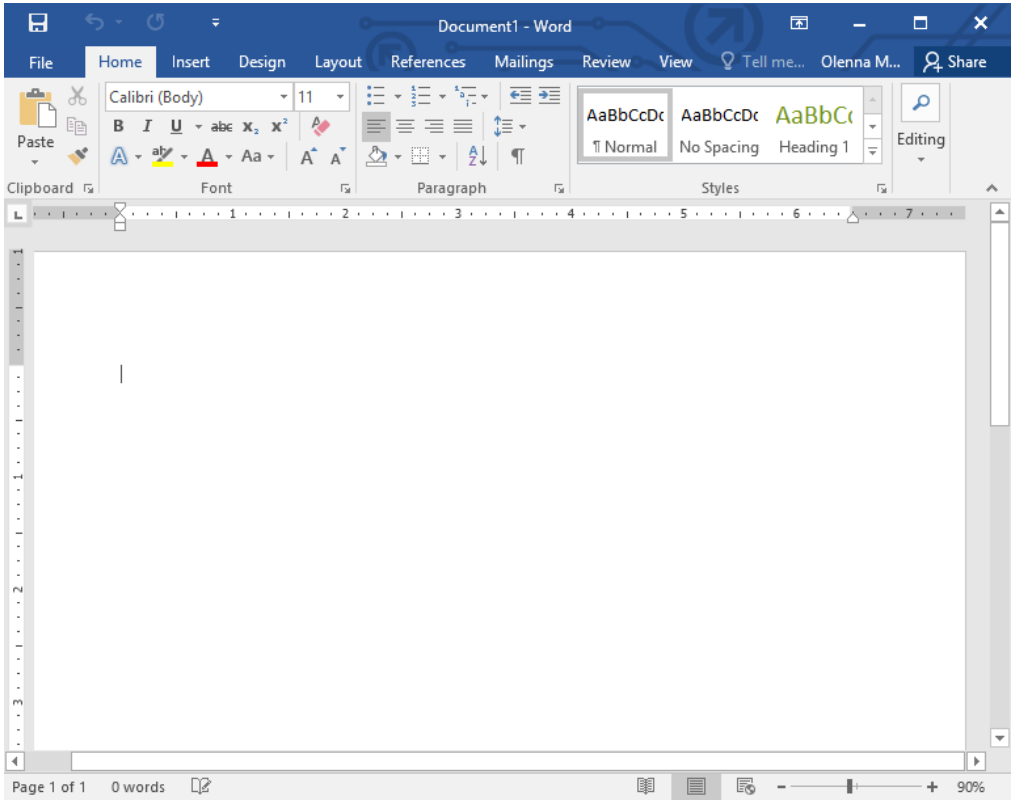
- **Los hilos no pueden ejecutarse por sí solos. Requieren la supervisión de un proceso padre para ser ejecutados.**

- **Dentro de un proceso puede haber uno o varios hilos en ejecución.**



1.2 Ejemplo

- Por ejemplo un proceso de la aplicación MS Word puede tener varios hilos en ejecución:
- Un hilo en *background* chequeando la gramática de lo que se escribe,
- Otro hilo puede estar guardando automáticamente los cambios del documento en el que se trabaja.



1.3 Single Thread

- Un **hilo o thread** es cada una de las tareas que puede realizar de forma simultánea una aplicación.

- Por defecto toda aplicación dispone de un único hilo de ejecución, al que se conoce como **hilo principal** o ***main thread***.

- Por defecto, al proceso que tiene **un solo hilo** en ejecución se le denomina ***single thread***

- Si dicha aplicación no despliega ningún otro hilo, sólo será capaz de ejecutar una tarea al mismo tiempo en ese hilo principal.

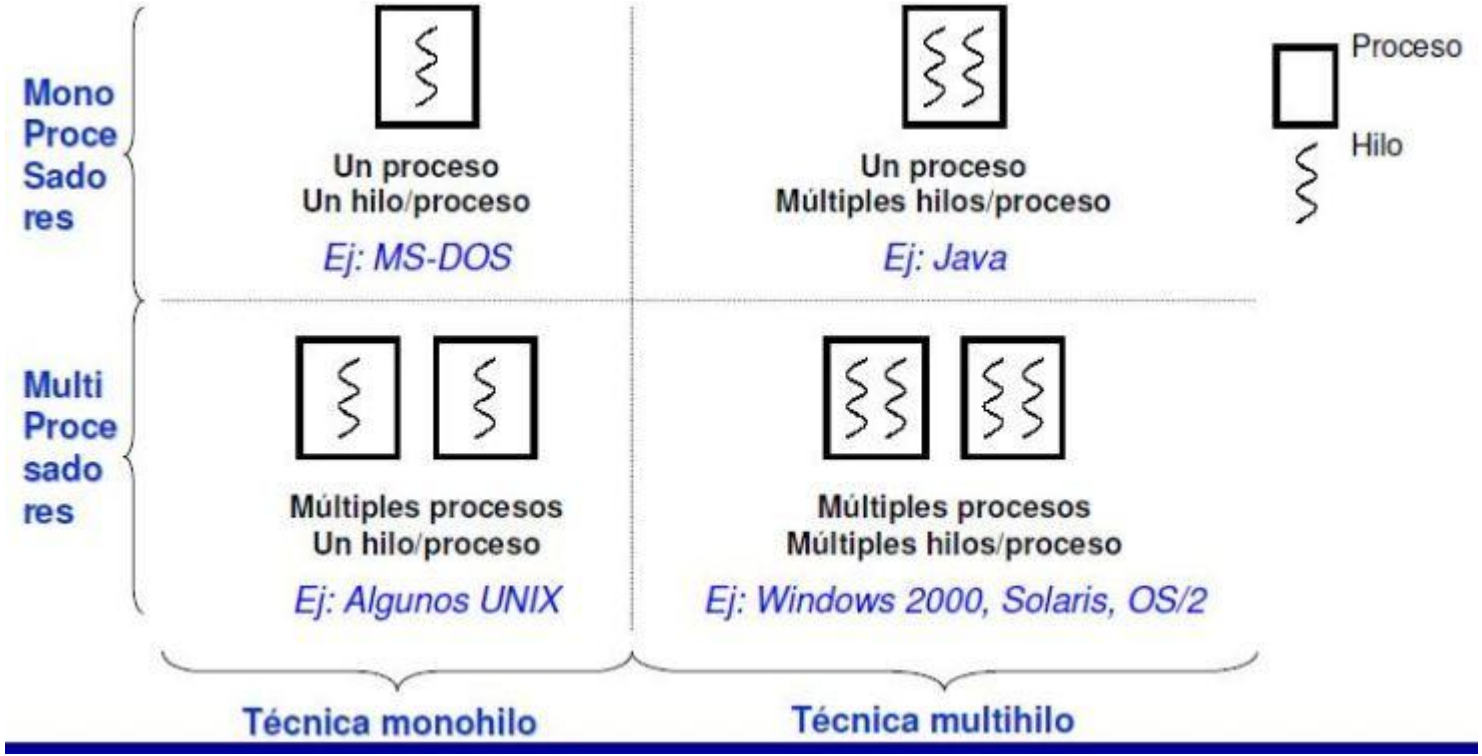
1.4 Multi Thread

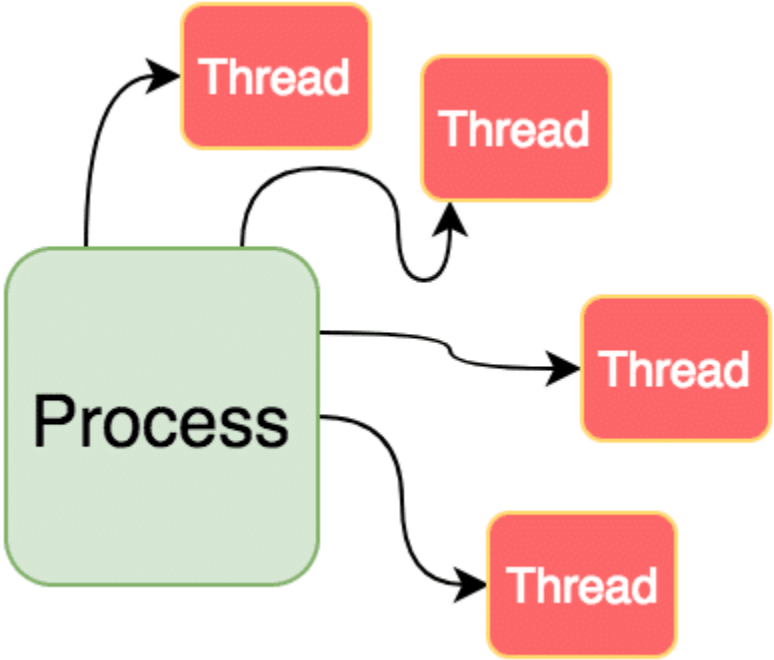
- **Para cada tarea adicional que se quiera ejecutar en un proceso se deberá lanzar un nuevo hilo o thread dentro del programa**

- **El hilo principal o *main thread* se encargará de lanzar los demás hilos de la aplicación, pudiendo estos a su vez lanzar nuevos hilos.**

- **A un programa que tiene varios hilos en ejecución se le denomina *multi-thread***

1.6 Tipos de ejecución de aplicaciones



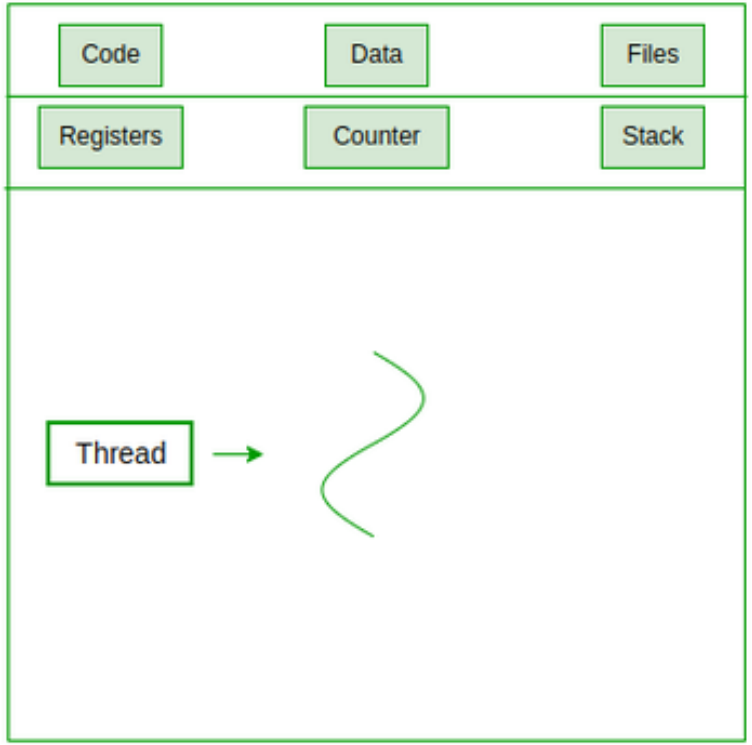


Hilos vs. Procesos

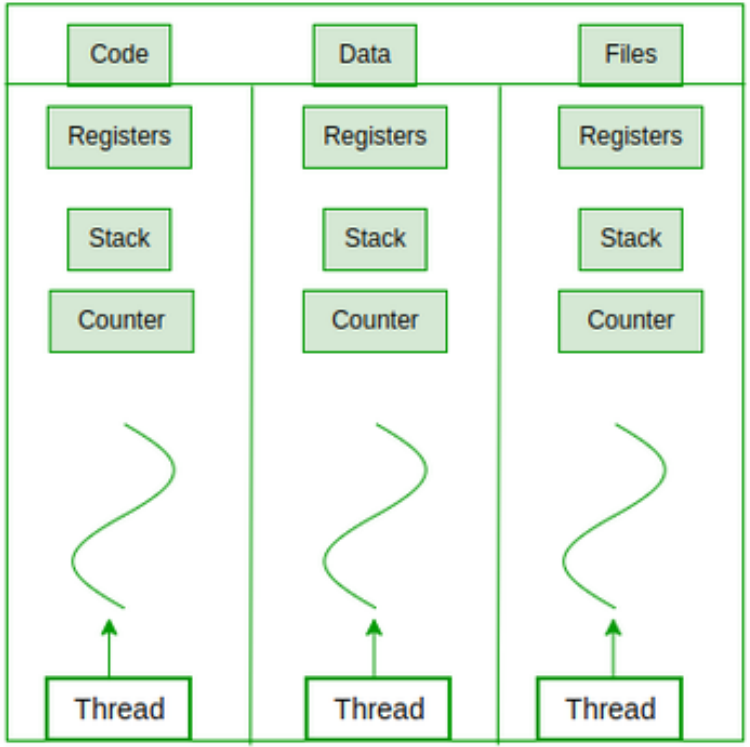
2.1 Uso de memoria

- **Un hilo es un concepto más *avanzado* que un proceso: cada proceso tiene su propio espacio en memoria mientras que en un hilo, el espacio es compartido (Por ejemplo: si abrimos 20 procesos cada uno de ellos consume 20x de memoria RAM, mientras que un hilo es mucho más ligero en memoria)**

2.1 Uso de memoria



Single Threaded Process



Multi Threaded Process

2.2 Ventajas de los hilos

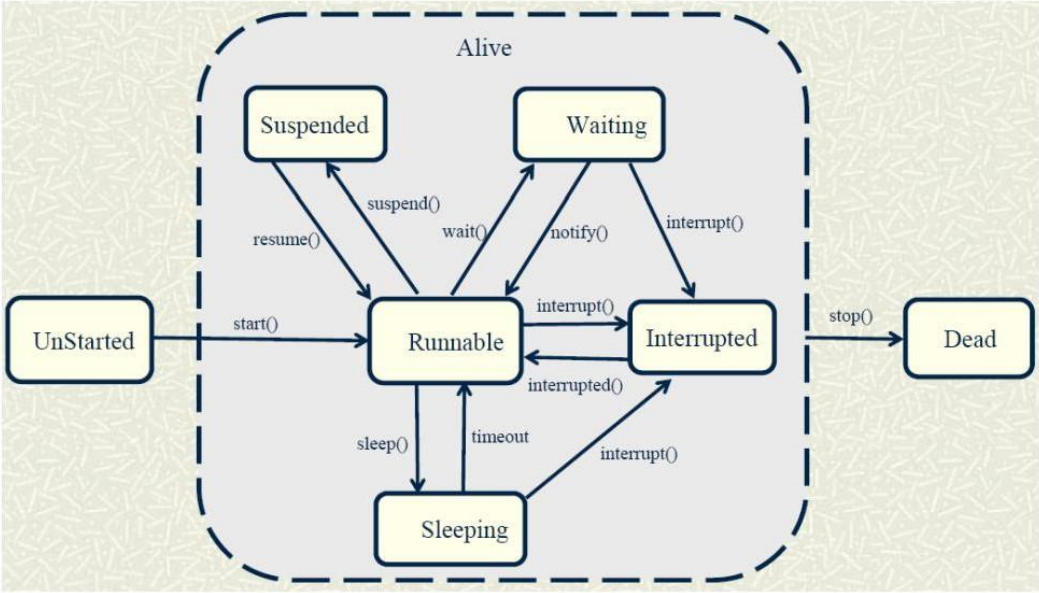
- **Un proceso no tiene acceso a los datos de otro procesos. Sin embargo un hilo sí accede a los datos de otro hilo.**

- **Los hilos proporcionan más de un camino de ejecución en un mismo programa.**

- **La programación multi-hilo en aplicaciones cliente-servidor agiliza los tiempos de retraso en la comunicación.**

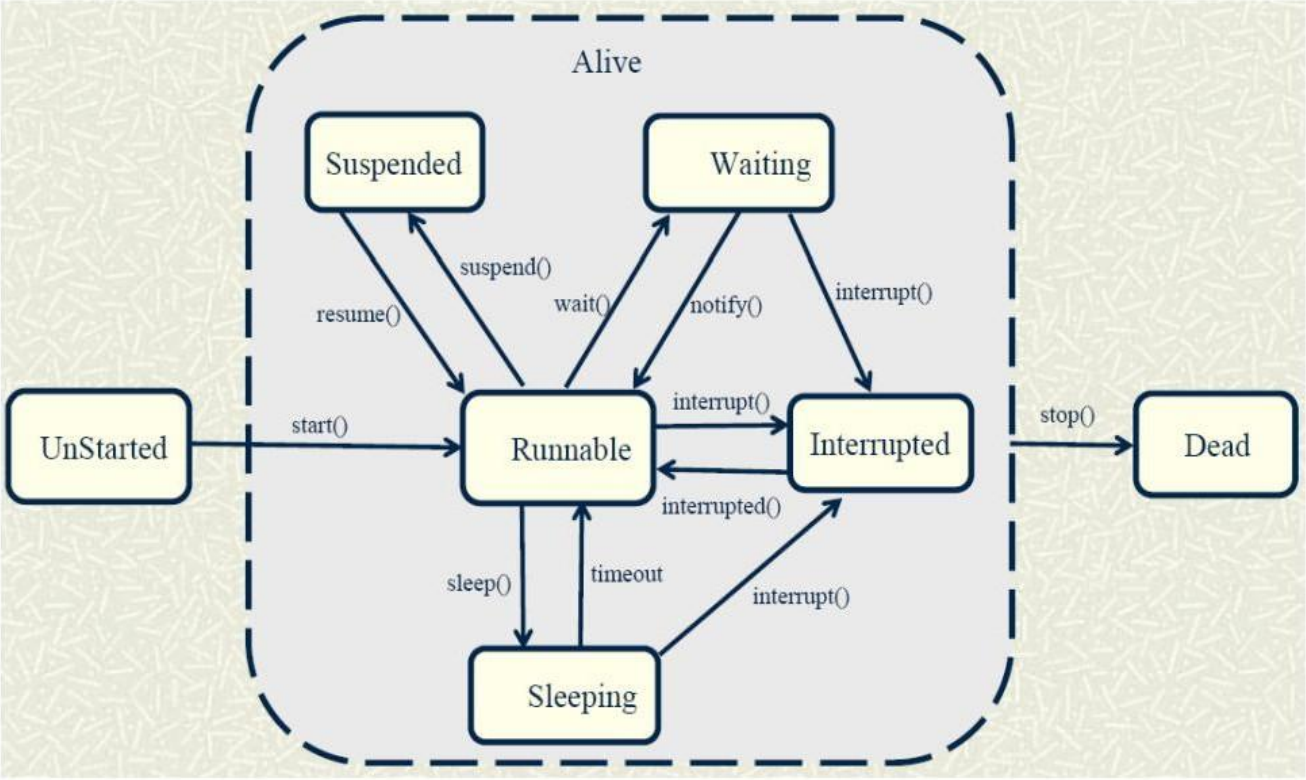
2.3 Ejecución de hilos

- Se considera una unidad básica de utilización de la CPU. Cada uno comprende:
  - Identificador de Thread
  - Contador de programa
  - Conjunto de registros
  - Pila
- Comparten con el resto de hilos del proceso:
  - Mapa de memoria (sección de código, sección de datos, shmem)
  - Ficheros abiertos
  - Señales, semáforos y temporizadores.



Estados de un hilo

3.1 Estados de un hijo en Java



3.1 Estados de un hijo en Java

El programador no es el encargado de controlar el estado de los procesos, lo hace el sistema operativo. Sin embargo un programa multihilo mal hecho puede dar lugar a un problema de interbloqueo:

- **Dead lock (abrazo mortal)**: Se produce cuando las peticiones y las esperas se entrelazan de forma que ningún proceso puede avanzar. Ningún proceso consigue hacer ninguna tarea útil y por lo tanto hay que esperar a que el administrador del sistema detecte el interbloqueo y mate procesos (o hasta que alguien reinicie el equipo).

3.2 Dead-lock







Ejecución de hilos en Java

4.1 Creación de hilos con “extends Thread”

```java
public class MiHilo extends Thread {
  @Override
  public void run() {
    for (int i = 0; i < 500; i++) {
      System.out.println("Soy un hilo y esto es lo que hago");
    }
  }
}
```

```java
public class LanzaHilos {
  public static void main(String[] args) {
    MiHilo tarea = new MiHilo();
    tarea.start();
  }
}
```

4.2 Creación de hilos con “implements Runnable”

```java
public class MiHiloRunnable implements Runnable {
  @Override
  public void run() {
    for (int i = 0; i < 500; i++) {
      System.out.println("Soy un hilo y esto es lo que hago");
    }
  }
}
```

```java
public class LanzaHilosRunnable {
  public static void main(String[] args) {
    MiHiloRunnable tarea = new MiHiloRunnable();
    Thread hilo = new Thread(tarea);
    hilo.start();
  }
}
```





Gestión y sincronización de hilos

5.1 Métodos estáticos de la clase Thread

Método	Descripción
currentThread()	Devuelve el objeto **Thread** que está ejecutándose en ese momento
sleep(long ms)	Pone el hilo en pausa el tiempo en milisegundos que le introduzcamos

5.2 Métodos de instancia de Thread

Método	Descripción
start()	Indica al intérprete de Java que cree un contexto del hilo del sistema y comience a ejecutarlo
suspend()	Provoca que se detenga la ejecución de un proceso sin destruir el estado de éste.
resume()	Reanuda un hilo suspendido
join()	Al utilizar este método nos aseguramos que no se ejecute otro hilo hasta que éste finalice
setPriority()	Asigna la prioridad al hilo indicada por el valor pasado como parámetro
getPriority()	Devuelve la prioridad del hilo de ejecución en curso
setName()	Asigna un nombre para identificar a los hilos de una forma más cómoda
getName()	Devuelve el nombre asignado
isAlive()	Devuelve “true” si el hilo está vivo

5.3 Métodos de sincronización de la clase Object

Nota: Estos tres métodos deben llamarse desde una zona sincronizada del código

Método	Descripción
wait()	Se coloca en una cola de espera de la que saldrá cuando otro thread, en la misma zona de acceso exclusivo, haga "notify()" o "notifyAll()".
notify()	Despierta a uno de los hilos que esté esperando por wait() a entrar en una zona sincronizada.
notifyAll()	Despierta a todos los hilos que estén esperando por wait() a entrar en una zona sincronizada.

END



chema.duran@iesmartinezm.es